CHOREO

算子专用编程语言介绍系列

实践篇之一：在 Topsop 中使用 Choreo 开发算子

Choreo 算子的产品集成

高性能计算应用的开发中，编程工具的选择至关重要。便利的编程工具不仅需要提供编写高性能计算程序的表达能力，还需要在系统集成、开发效率和调试体验方面满足复杂项目的需求。Choreo 正是这样一门计算核编程工具。它不仅实现了并行编程能力的零成本抽象，还被设计为一个独立的编译器模块，能够按需生成标准的 C 模块或目标系统的源码。这种设计也保证了 Choreo 作为编程工具的可维护性和集成能力。

这一次，我们会使用在《编程篇之一》文中使用到的动态矩阵乘算子为例，展示如何将使用 choreo 实现的算子集成到 Topsop 算子库。

```
__co__ auto matmul(s32 [M, K] lhs, s32 [K, N] rhs) { ... }

int main() {
  auto lhs = choreo::make_spandata<choreo::s32>(128, 256);
  auto rhs = choreo::make_spandata<choreo::s32>(256, 256);
  // ...

  auto res = matmul(lhs.view(), rhs.view());
  // ...
}
```

这里我们给出了上次定义的 matmul *Tileflow* 函数的调用部分。在这个名为 `matmul` 的算子中，我们声明张量参数 `lhs` 与 `rhs` 的形状分别为 `[M, K]` 与 `[K, N]`。这里的 `M, K, N` 是运行时提供的值。而在函数的调用部分，我们使用了 choreo 的 API `choreo::make_spandata` 来声明了一块**有形状的**数据。并且我们我们调用名为 `.view()` 成员函数将两块数据指针传入 *Tileflow* 函数中进行处理。

读者可能好奇这里代码是如何连接的，我们可以通过 `choreo -es` 命令来观察下：

```
choreo::spanned_data<choreo::s32, 2>
matmul(const choreo::spanned_view<choreo::s32, 2> & lhs,
       const choreo::spanned_view<choreo::s32, 2> & rhs) {...}
```

可以看到，choreo 生成的入口函数是一个**主控函数**，它接受两个`choreo::spanned_view`参数。，这与现有许多算子实现使用**设备函数**作为接口并不相同。不过通过这样的方式，choreo 实现了运行时的形状检查能力，而 choreo 的动态形状也很依赖于所生成的主控代码部分的功能。事实上，细心的读者可能还观察到 choreo 生成的设备入口函数原型为：

```
__global__ void matmul(int * lhs, int * rhs, int * output,
                       unsigned K, unsigned M, unsigned N) {}
```

这里形状值`M, K, N` 被传入了设备函数中，以实现动态形状相关的处理。

因此在 choreo 与 topscc 代码的集成中，应当总是使用 choreo 提供的功能函数，如 `choreo::make_spanview`, `choreo::make_spandata` 等来实现数据的传递。

计算核编程工具设计对比

现有的编程工具通常基于几种常见的设计进行开发，除了 Choreo 采用的独立编译器模式之外，常见的设计有：

- **编程 API 库** 通过宿主语言（如 C++/Python）扩展语义描述计算逻辑，代表方案包括 Tensor Comprehension、TVM。其工作流程依赖宿主编译器前端进行语法解析，生成中间表达后绑定 LLVM 后端。

  痛点：工具实现体量很大，部署集成较为臃肿。需要与后端直接绑定/集成，API 前端与编译器后端需严格版本匹配（比如 TVM 0.8 前端仅兼容 LLVM 12）
- **宿主语言扩展 eDSL**：通过编译器插件扩展语法（如 CUDA 的 __global__、OpenMP 的 #pragma omp），直接在宿主语言中表达并行语义。典型代表包括 NVIDIA CUDA、Intel ISPC。

  痛点：工具链深度绑定（如 CUDA 需 nvcc），调试需切换专用工具（nsight/cuda-gdb）
- **基于元编程的模板库（包括纯头文件库**：通过模板特化实现编程接口设计，代表方案为 NVIDIA Cutlass、oneDNN、Eigen、Xbyak 等。基于 C++ 模板元编程在编译期展开 Expression Tree，实现编译期优化。

  痛点：模板实例化导致二进制体积显著增加，编译时间随计算复杂度指数增长，调试符号膨胀

Choreo

Topsop 中 Choreo 算子的集成方法

虽然使用 `choreo::make_spanview`, `choreo::make_spandata` 并不复杂，但为了方便在现有项目中集成 choreo 代码，我们为 Topsop 提供了向 choreo 带形状数据的功能转换接口，比如将 topsop 中的 `topsopTensorHandle_t` 类型的数据转换成 choreo 函数可以使用的 `choreo::spanned_view` 类型，下面的代码即是一个例子：

```
topsopStatus_t topsatenIdentity(topsopTensorHandle_t output,
                                const topsopTensorHandle_t lhs,
                                const topsopTensorHandle_t rhs,
                                const topsopSize_t size,
                                const topsopDataType_t data_type,
                                topsStream_t stream) {
  // Sanity Checks ...

  std::unique_ptr<OpAtenIdentity> p_impl = std::make_unique<OpAtenIdentity>(
      output.to_choreo_spv(), // convert from topsopTensorHandle_t to choreo::spanned_view
      lhs.to_choreo_spv(),
      rhs.to_choreo_spv(),
      size,
      data_type,
      stream);

  FLAME_TRACE_INFO_START(IdentityOp, p_impl)

  // Post exec checks ...
  status = p_impl->CheckArgs();
  return status;
}
```

可以看到，调用接口 `.to_choreo_spv()` 即可实现该功能。

在构建系统部分，我们可以采用标准的分段式编译构建的模式来将 choreo 集成到 Topsop 的 CMAKE 配置中。这里我们采用将 choreo 代码独立编译，即先将`.co`中的算子编译为标准的二进制文件模块，再与其它已有的 Topscc 模块进行的链接的方式完成。具体的实现方式如下所示：

```
add_custom_command(
    OUTPUT matmul.o
    COMMAND
      ${CHOREO_COMPILER}
      -c -fpic
      ${CMAKE_CURRENT_SOURCE_DIR}/matmul.co -o matmul.o
    DEPENDS matmul.co
    COMMENT "Compiling matmul.co with Choreo"
)

op_add_library(kernel_matmul matmul_host.tops)
add_sources_cflags(kernel_matmul matmul_host.tops gcu300)
target_link_libraries(
    kernel_matmul
    PRIVATE
    ${CMAKE_CURRENT_BINARY_DIR}/matmul.o
)
```

这里将`${CHOREO_COMPILER}`设置为系统中 choreo 编译器的位置即可。读者可以看到，Choreo 与宿主语言 topscc 的集成是极为简单的。而相较于其它的（提供 CMake 配置或作为外部项目集成的）计算核编程工具而言，Choreo 对当工具链版本没有强绑定关系。这是 choreo 的一个优点所在。

【注】spanned_view与spanned_data

`choreo::make_spanview`, `choreo::make_spandata` 会分别产生`choreo::spanned_view`, `choreo::spanned_data` 类型。前者是 choreo *Tileflow* 函数的输入类型，不过其并不管理所指向的数据；而后者则使用智能指针管理(`std::unique_ptr`)所指向的数据。其可以通过成员函数`.view()`来产生`choreo::spanned_view`对象以供 choreo *Tileflow* 使用。

深入了解 Choreo

如果想深入了解 Choreo，我们准备了相关的文档、视频教程供您学习：

- 网页教程：http://choreo.enflame.cn/
- 视频教程：https://learning.enflame-tech.com/course/144

参与到 Choreo 的开发中

无论是汇报问题，或是建议新的 Choreo 特性，都可以通过在 Choreo 项目主页创建 issue 的方式达成：http://git.enflame.cn/xiaofeng.guan/choreo/-/issues

Choreo 的**项目主页**在：http://git.enflame.cn/xiaofeng.guan/choreo，其包含了编译使用 Choreo 的说明

如有需要请联系：

- 官孝峰：xiaofeng.guan@enflame-tech.com
- 石恒：heng.shi@enflame-tech.com

CHOREO

欢迎加入 *Choreo* 服务群，汇报 *Choreo* 的问题，分享试用体验，以及提出建议